
# CSEE 4280 - Advanced Digital Design

Final Project: Alphabet Soup

Group 11: Aubrey David and Hannah Ho

04/25/25

# Table of Contents

# Final Code

The final code can be found in the Group 11 Github repository under tree/main/FinalCryptoAcc.

# 1. Introduction

AES or Advanced Encryption Standard is a standard chosen by the U.S. government to store classified information in a way that is not easily decipherable to hackers. Group 11 created the Alphabet Soup (Cryptographic Accelerator) Project to encrypt sensitive data using a secret key that is combined with a keyboard input from a USB-UART PMOD to an FPGA NEXYS A7 board. The encryption process consists of manipulating the 128-bit text through substituting bytes, shifting rows, mixing columns, and adding to an expanded round key. The message will be encrypted or decrypted based on user input through up and down buttons on the FPGA board and this will all be displayed through the VGA output. Additionally, encryption and decryption rounds are broken down into stages to maximize throughput.

The hierarchical design of Group 11's project will consist of a keyboard input and an output displayed on a monitor. The diagram describing how the keyboard input connects to the display monitor can be found in Figure 1.1.

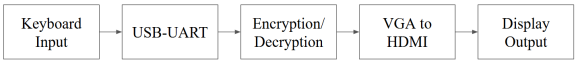


**Figure 1.1:** Overall System Diagram

The project breakdown is divided into 3 main subsections: USB-UART, AES, and VGA. All figures can be found zoomed-in in Appendix A.

# 2. USB-UART

To begin, this project uses USB-UART communication to receive an input text. While the Nexys A7 already has UART compatibility, Group 11 chose to utilize the PMOD pins while receiving serial input. The PmodUSBUART module, distributed by Digilent, allows the PMOD pins to connect to a micro USB cable [8]. This cable, in turn, connects to a laptop's keyboard. Serial communication is driven from the laptop using PuTTy. In order for UART to work, the transmitting and receiving devices must agree on a baud rate, data word size, and stop bit size [7]. These values are 9600, 8, and 1 respectively for this project, and were configured separately on PuTTy and the UART receiver module.

$$baud = 9600$$
$$sampling\ rate = baud * 16 = 9600 * 16 = 153600$$
$$counter\ limit = clock\ speed/sampling\ rate$$
$$= 100MHz/153600 = \sim 651$$
$$counter\ bits = log_2(counter\ limit) = log_2(651) = 10$$

**Figure 2.1:** Baud parameter calculations

The UART top module (**UART.v**) instantiates the baud rate generator, receiver, and FIFO generator modules. This module is parameterized to fit this project's required baud rate, along with dictating how much data the FIFO can hold. Parameterization allows such variables to change easily without writing a lot of new code. Based on the FPGA's clock speed and desired baud rate, a sampling rate, counter limit, and a counter bit size are calculated [10], as seen in Figure 2.1. The number of FIFO addresses corresponds to its size, so setting it to 2^4 = 16 total addresses for 128 bits [10]. These addresses are each 8 bits wide to accommodate ASCII characters.

## Baud Rate Generator

The baud rate generator module (**baud_rate_generator.v**) takes the baud rate parameters and converts the FPGA clock signal into an oversampling tick signal that the UART receiver uses [10]. This is good practice for noise immunity.

## UART Receiver

The UART receiver module (**uart_receiver.v**) acts as a shift register, taking in individual data bits and outputting a reformatted byte [10]. It waits in an "Idle" state until the data line goes LOW, which is indicative of a start bit, and transitions to "Start". After 7 ticks, roughly the middle of the start bit, it transitions to "Data" and the consecutive data bits are shifted and stored in a register by estimating their halfway points (15 ticks each). Once the parameterized size of data bits is reached, the receiver transitions to "Stop" and waits the number of ticks corresponding to the number of stop bits (16 for 1 stop bit) before returning to "Idle". The stored bits are outputted as a byte to be buffered in the FIFO.

## FIFO

The FIFO module (**fifo.v**) is a synchronous first-in, first-out memory used to store the input from the UART before sending it to the AES modules. It has a flexible size from its parameterization, allowing it to store all 128 bits required from this project. It contains ports for reading data in and out, along with full and empty signals to prevent writing to it while full or reading from it while empty. It accesses each byte from within using internal addressing. On the rising edge of the clock, if writing is enabled and the FIFO is not full, data will be stored at the current write pointer. Likewise, if reading is enabled and the FIFO is not empty, data will be taken out at the current reading pointer. If reading and writing are both enabled, both functions occur at once and the pointers update accordingly, which allows for efficient data flow. Since the data size is small enough, synthesizing this makes it LUT-based memory by default.

# 3. AES

After receiving the input text, the program decides whether to encrypt or decrypt the text after **KeyExpansion.v** expands a 128-bit key into 10 128-bit keys for each round of encryption. This module expands the key by using an initial key and inputting it into a 'g' function [1]. The last word of the inputted key is used in the 'g' function to create a new word by rotating bytes left, substituting with a universal value from the S-Box table, and finally XOR combining with a predetermined round constant. This new last word is combined with the first word of the 128-bit input and the new first word is combined with the second word of the 128-bit input. The diagram for the key expansion logic from Neso Academy is found in Figure 3.1 [1].

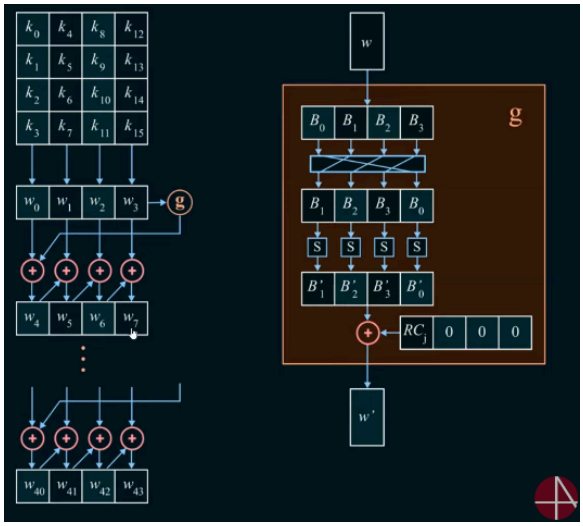

**Figure 3.1:** Key Expansion Logic

The orange box represents the ‘g’ function and the pink ‘+’ represents XOR operations. The KeyExpansion.v module calls the ‘g’ function followed by the 4 XOR operations to create a new 128-bit key. A top module, KeyExpansionTop.v, calls KeyExpansion.v 10 different times to create 10 different keys (**key1 - key10**) with each key using the previous one to create a new key. After 11 keys are created, including the initial key (**key**), they are sent to Encryption.v or Decryption.v along with the input text.

## Encryption

The AddRoundKey.v module is firstly called to XOR combine the input text with the original **key** as shown in Figure 3.2.

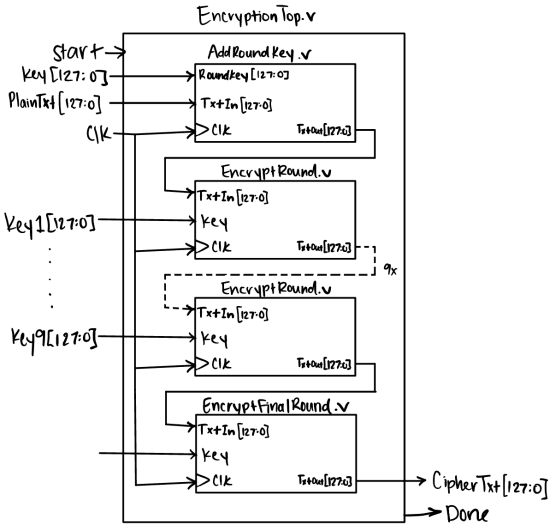


**Figure 3.2:** EncryptionTop.v Block Diagram

Next, 10 rounds of encryption take place where the first 9 rounds consist of substituting bytes, shifting rows, mixing columns, and adding round key as shown in Figure 3.3.

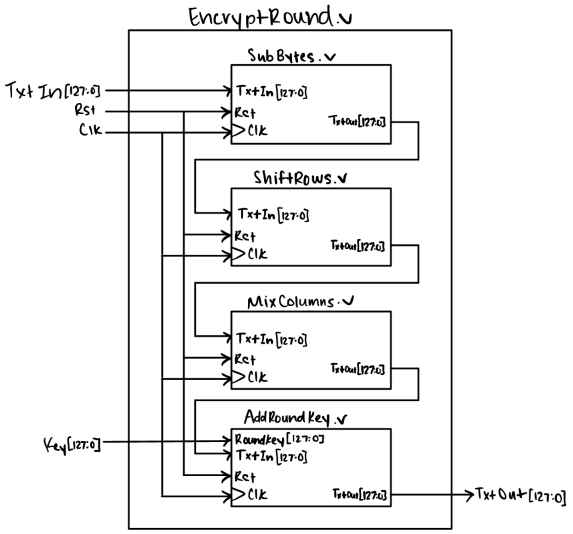


**Figure 3.3:** EncryptRound.v Block Diagram

The last round of encryption consists of the same modules minus MixColumns.v as shown in Figure 3.4.

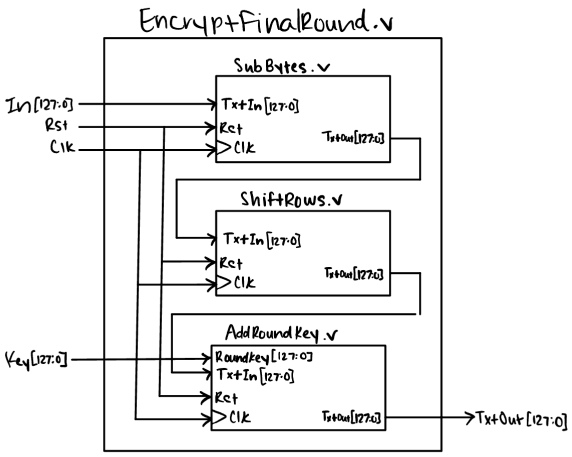


**Figure 3.4:** EncryptFinalRound.v Block Diagram

Each of the encryption round modules do a different manipulation of the plain text in order to cipher it.

## Substituting Bytes

In SubBytes.v, every byte is replaced with the byte that intersects the row and column nibble value using the S-Box table in Figure 3.5 [2].

| | | Y | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | a | b | c | d | e | f |
| X | 0 | 63 | 7C | 77 | 7B | F2 | 6B | 6F | C5 | 30 | 01 | 67 | 2B | FE | D7 | AB | 76 |
| | 1 | CA | 82 | C9 | 7D | FA | 59 | 47 | F0 | AD | D4 | A2 | AF | 9C | A4 | 72 | C0 |
| | 2 | B7 | FD | 93 | 26 | 36 | 3F | F7 | CC | 34 | A5 | E5 | F1 | 71 | D8 | 31 | 15 |
| | 3 | 04 | C7 | 23 | C3 | 18 | 96 | 05 | 9A | 07 | 12 | 80 | E2 | EB | 27 | B2 | 75 |
| | 4 | 09 | 83 | 2C | 1A | 1B | 6E | 5A | A0 | 52 | 3B | D6 | B3 | 29 | E3 | 2F | 84 |
| | 5 | 53 | D1 | 00 | ED | 20 | FC | B1 | 5B | 6A | CB | BE | 39 | 4A | 4C | 58 | CF |
| | 6 | D0 | EF | AA | FB | 43 | 4D | 33 | 85 | 45 | F9 | 02 | 7F | 50 | 3C | 9F | A8 |
| | 7 | 51 | A3 | 40 | 8F | 92 | 9D | 38 | F5 | BC | B6 | DA | 21 | 10 | FF | F3 | D2 |
| | 8 | CD | 0C | 13 | EC | 5F | 97 | 44 | 17 | C4 | A7 | 7E | 3D | 64 | 5D | 19 | 73 |
| | 9 | 60 | 81 | 4F | DC | 22 | 2A | 90 | 88 | 46 | EE | B8 | 14 | DE | 5E | 0B | DB |
| | a | E0 | 32 | 3A | 0A | 49 | 06 | 24 | 5C | C2 | D3 | AC | 62 | 91 | 95 | E4 | 79 |
| | b | E7 | C8 | 37 | 6D | 8D | D5 | 4E | A9 | 6C | 56 | F4 | EA | 65 | 7A | AE | 08 |
| | c | BA | 78 | 25 | 2E | 1C | A6 | B4 | C6 | E8 | DD | 74 | 1F | 4B | BD | 8B | 8A |
| | d | 70 | 3E | B5 | 66 | 48 | 03 | F6 | 0E | 61 | 35 | 57 | B9 | 86 | C1 | 1D | 9E |
| | e | E1 | F8 | 98 | 11 | 69 | D9 | 8E | 94 | 9B | 1E | 87 | E9 | CE | 55 | 28 | DF |
| | f | 8C | A1 | 89 | 0D | BF | E6 | 42 | 68 | 41 | 99 | 2D | 0F | B0 | 54 | BB | 16 |

**Figure 3.5:** S-Box

## Shifting Rows

In ShiftRows.v, the data is organized in a 4x4 matrix where the values of the input text **txt**[127:0] are organized in Table 3.6.

| | | | |
|---|---|---|---|
| txt[127:120] | txt[95:88] | txt[63:56] | txt[31:24] |
| txt[119:112] | txt[87:80] | txt[55:48] | txt[25:16] |
| txt[111:104] | txt[79:72] | txt[47:40] | txt[15:8] |
| txt[103:96] | txt[71:64] | txt[39:32] | txt[7:0] |

**Table 3.6:** Matrix Organization

From this table, the values are shifted to the left by an increasing amount shown in Figure 3.7 [3].

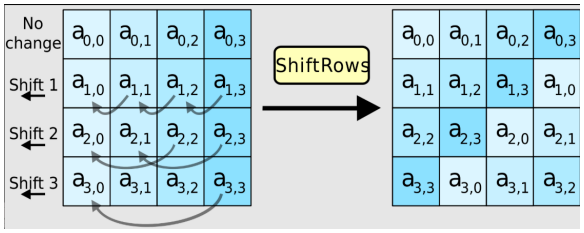


**Figure 3.7:** Shift Rows Logic

## Mixing Columns

In MixColumns.v, each column of the 128-bit data is modulo multiplied with Rijndael's Galois Field. The diagram for the multiplication is shown in Figure 3.7 [4].

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} a_{11} \\ a_{21} \\ a_{31} \\ a_{41} \end{bmatrix} = \begin{bmatrix} b_{11} \\ b_{21} \\ b_{31} \\ b_{41} \end{bmatrix}$$

...

$$\begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{bmatrix}$$

**Figure 3.7:** Mix Columns Logic

This entails each element is multiplied by either 1, 2, or 3 in that predetermined order. Multiplication by 1 results in the same value, so functions inside the module were created to multiply by 2 and 3. To multiply by 2, a value is shifted left once and XORed with 1 if the original value's MSB was 1. To multiply by 3, the multiply by 2 function is called in addition to being XORed with the original value.

To ensure each element of the matrix is multiplied by the correct number, the output for this module is built byte-by-byte. Each specific segment of the 128-bit data is called into the necessary function.

Finally, a summy FSM diagram showing the flow of how the input flows through the modules is shown in Figure 3.8.

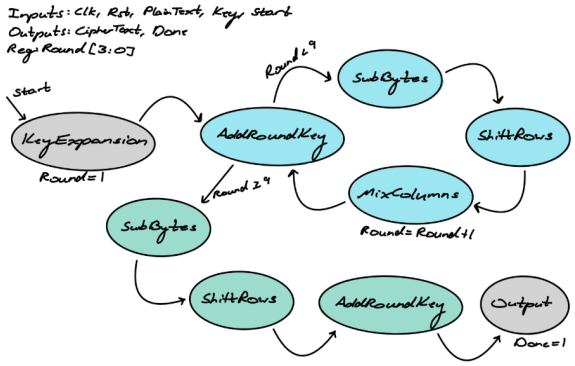


**Figure 3.8:** FSM Diagram for Encryption.v

## Decryption

Decrypting text works in a similar fashion to encryption but uses inverse manipulation modules in a different order as shown in Figure 3.9.

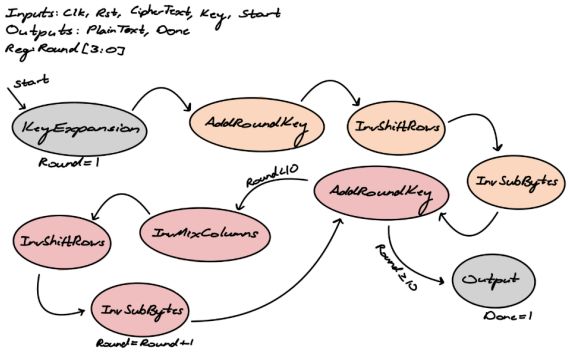


**Figure 3.9:** FSM Diagram for Decryption.v

The inverse modules work similar to the encryption modules. InvShiftRows.v shifts the rows to the right a certain number of times (Appendix A). InvSubBytes.v replaces bytes from the input text with bytes from the inverse S-Box (Appendix A). Finally, InvMixColumns.v modulo multiples by the matrix shown in Figure 3.10.

$$\begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix}$$

**Figure 3.10:** Inverse Mix Columns Matrix

# 4. VGA

The purpose of the VGA is to output to a display monitor for the user to see what is happening. Group 11 accomplishes this by using a VGA controller, ASCII ROM, and Text Generator [6]. The VGA controller takes a clock input to control hsync and vsync along with position and video_on values [6]. The ASCII ROM is a dictionary of addresses that map ASCII values to an 8x16 bit matrix that correlates with what the values should look like on the display. Finally the text generator ties all of the

components together to output RGB values to the screen as shown in Figure 4.1 [6].

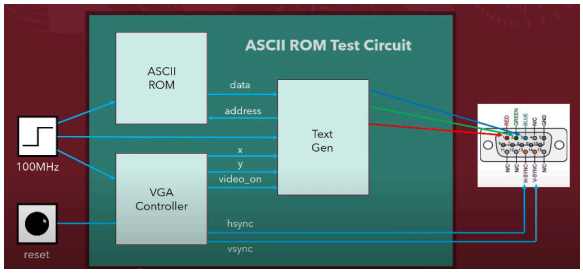


**Figure 4.1:** VGA Output Block Diagram

Using previously created vga controller and text generator from FGPA Jeff, Group 11 adds to his initial idea [11]. Instead of just printing out a predetermined word from a text file, Group 11 takes a 128-bit input from AES.v to extract one byte at a time, find the correlating ascii value in ascii_rom.v, and print it to the monitor. The vga_prompt.v module simply displays the prompt with pre-called ascii values that the VGA can display to ask the user to input a 16 character message.

# 5. Top Module

The top module (**Top.v**) takes input from the FPGA in the form of two debounced buttons for selecting an AES algorithm and the serial Rx input from the PmodUSBUART. It outputs to an LED to alert the user of valid data input and to the VGA to display results. In addition to instantiating everything, the top module also handles the program flow and buffering from the FIFO memory into a register the AES module can read from.

## Program Flow

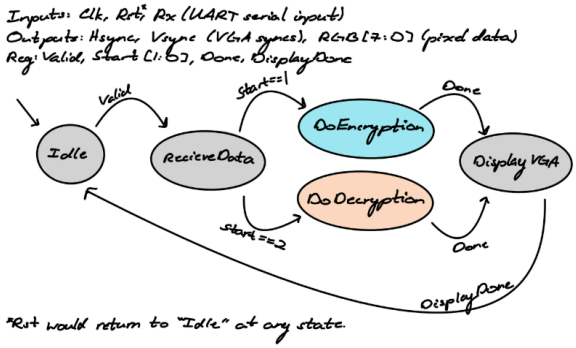


**Figure 5.1**: FSM diagram for Top.v

Figure 5.1 shows the planned FSM for this module. This is closely followed in the actual implementation; The program idly waits to receive a full FIFO from the UART input. Once this is valid, the data is received from the FIFO and prepared to be sent to the AES module. The user selects the algorithm with a button press to begin an encryption or decryption stage, and then automatically sends this result to be displayed on the VGA monitor. The largest change is that the user must send a reset signal to return to the original idle state.

## FIFO Buffering

The top module also handles fetching the UART input from memory and preparing it for AES encryption or decryption. Once the FIFO has been sensed as full, its memory is read out byte-by-byte into an internal 128-bit register, which can then be used as AES input.

## Button Debouncing

Also instantiated in this module is a simple debouncing module (**debounce_explicit.v**). This is so that multiple signals are not sent to start the AES module at once, which could potentially cause issues

for properly processing the UART input. It uses an FSM to wait and see if a button press is long enough to be considered valid, and outputs a high pulse when that happens for each button.

# 6. Design Evaluation

| Resource | Utilization | Available | Utilization % |
|---|---|---|---|
| LUT | 17919 | 63400 | 28.26 |
| LUTRAM | 6 | 19000 | 0.03 |
| FF | 660 | 126800 | 0.52 |
| BRAM | 0.50 | 135 | 0.37 |
| IO | 20 | 210 | 9.52 |
| BUFG | 3 | 32 | 9.38 |

**Figure 6.1**: Resource utilization

Based on the resource utilization in Figure 6.1, this project works well within what is available on the Nexys A7. As predicted, the majority of memory is stored in LUT RAM, which is commonly done on FPGAs for relatively small memory sizes. From this, it can be assumed that this project runs very efficiently due to its pipelined AES encryption.

From working on it, the time overhead of generating this project's bitstream is found to be significant. This is a consequence of pipelining, but sacrificing time to upload is worth it when the encryption and decryption algorithms work near-instantly.

# 7. Testing

Each encryption, decryption, and top modules have individual self-checking test benches that confirm if the output is aligned. To accomplish this, a step by step AES encryption simulator is used to find what the expected 128-bit value should be at every step of encryption and decryption [7]. This value is then compared with the output value from the module. The test bench not only creates a GTK waveform to see what the values are but the **run.log** file will show how many errors result from the test module and what the errors are. An example output from InvShiftRows_tb.v is shown in Figure 7.1.

```
LXT2 info: dumpfile dump.lx2 opened for output.
Test done,           0 errors

InvShiftRows_tb.v:51: $finish called at 4000 (1ps)
```

**Figure 7.1:** Output Testbench from InvShiftRows_tb.v

Additionally, when working with the USB-UART PMOD, LED's on the FPGA board were used to indicate if the board received 128-bits of data and puTTy was used to display the text as the user is typing it. For testing the VGA, Group 11 started using a text file to display correlating ascii characters [1]. These VGA modules gradually progressed to displaying characters from the module for the prompt and finally displaying a 128-bit character input from the AES-core.

# 8. Future Improvements

At this point, the user is able to encrypt and decrypt any text they type but it becomes difficult to encrypt or decrypt text that is previously encrypted or decrypted. After a plain text is encrypted, the cipher text is outputted as a 128-bit hex number that cannot always be displayed in ASCII characters. One possible solution to this is to display the cipher text in base-64 where the cipher text will consist of numbers, letters, '=', '/'. and '+.' The user can then type this base-64 ciphered text and Group 11 would need to alter the program to process the data in

base-64 format and finally return back to hex to display the ascii value. Another solution consists of only allowing the user to input a plain text to be encrypted and in order to decrypt the text the user can press BTND to get the plain text back. Finally, another improvement is to show what the user types as the user is typing it.

# 9. Author Contribution

Each author's contribution is shown below where the correlating authors for each module also made the test benches, researched, and contributed to the report for that corresponding module/section.

## Aubrey:

**UART:**

- Top.v
- UART.v
- baud_rate_generator.v
- uart_reciever.v
- fifo.v
- debounce_explicit.v
- constraint.xdc

**AES:**

- MixColumns.v
- Top.v
- AES.v
- InvMixColumns.v
- InvShiftRows.v

## Hannah:

**AES:**

- AddRoundKey.v
- KeyExpansion.v
- EncryptionTop.v
    - ShiftRows.v
    - SubBytes.v
    - Round/FinalRound.v
- DecryptionTop.v
    - InvSubBytes.v
    - Round/FinalRound.v

**VGA:**

- vga_prompt.v
- vga_test.v
- vga_sync.v [11]
- textGeneration.v [11]
- ascii_rom.v
- constraint.x

# 10. Project Checkoff

The project checkoff, including what was accomplished in the commitment, goal, and stretch, is found below.

## The Commitment: *Completed*

- **PMOD:** A USB-UART PMOD is used with a keyboard for the user to type a message.
- **Memory:** The UART uses inferred BRAM with FIFO configuration and the VGA uses ROM for ascii character font.
- **VGA:** The encrypted/decrypted 128-bit hex message is displayed on a monitor through the VGA output.
- **AES Encryption/Decryption:** EncryptionTop.v converts a plain message into a ciphertext and DecryptionTop.v converts a ciphertext into a plain message.

## The Goal: *Partially Completed*

- **More Info To User:** There is a prompt to ask the user to input a message to encrypt/decrypt
- **User Input on Keystroke:** Not completed

## The Stretch: *Not Completed*

- **Changing encryption keys through switches:** Not completed

- **More efficient:** Not Complet

# 11. References


[1] https://www.youtube.com/watch?v=0RxLUf4fxs8

[2] https://www.redalyc.org/journal/5122/512253718012/html/

[3]https://crypto.stackexchange.com/questions/102946/why-aes-code-is-working-even-with-3-rows

[4] https://fpga.mit.edu/videos/2023/team19/report.pdf

[5]https://medium.com/quick-code/understanding-the-advanced-encryption-standard-7d7884277e7

[6] https://www.youtube.com/watch?v=eD4EyVq42mQ

[7]https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18842446/Setup+a+Serial+Console?view=blog

[8]https://digilent.com/reference/_media/pmod:pmod:pmodusbuart_rm.pdf?srsltid=AfmBOopryfOeyO3FfmaqMSgWq-SYuQpSXrVpy0WMYmlffBobWHIV1MnK

[9]https://github.com/FPGADude/Digital-Design/tree/4cb93eeaba434eb02c2e200060921fe0e5aebf03/FPGA%20Projects/UART

[10]https://www.youtube.com/watch?v=L1D5rBwGTwY&ab_channel=FPGADiscovery%28LearningHowtoWorkwithFPGAs%29

[11] https://github.com/klam20/FPGAProjects/tree/main/VGATextGeneration


# Appendix A

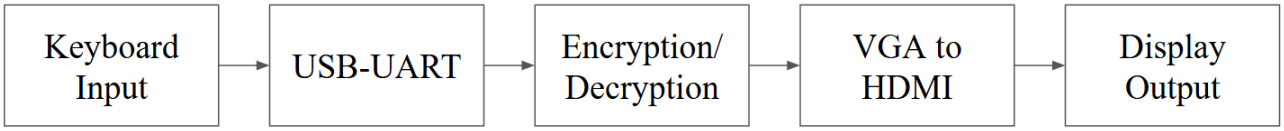


**Figure 1.1:** Overall System Diagram

$$baud = 9600$$

$$sampling\ rate = baud * 16 = 9600 * 16 = 153600$$

$$counter\ limit = clock\ speed/sampling\ rate = 100MHz/153600 = \sim 651$$

$$counter\ bits = log_2(counter\ limit) = log_2(651) = 10$$

**Figure 2.1:** Baud parameter calculations

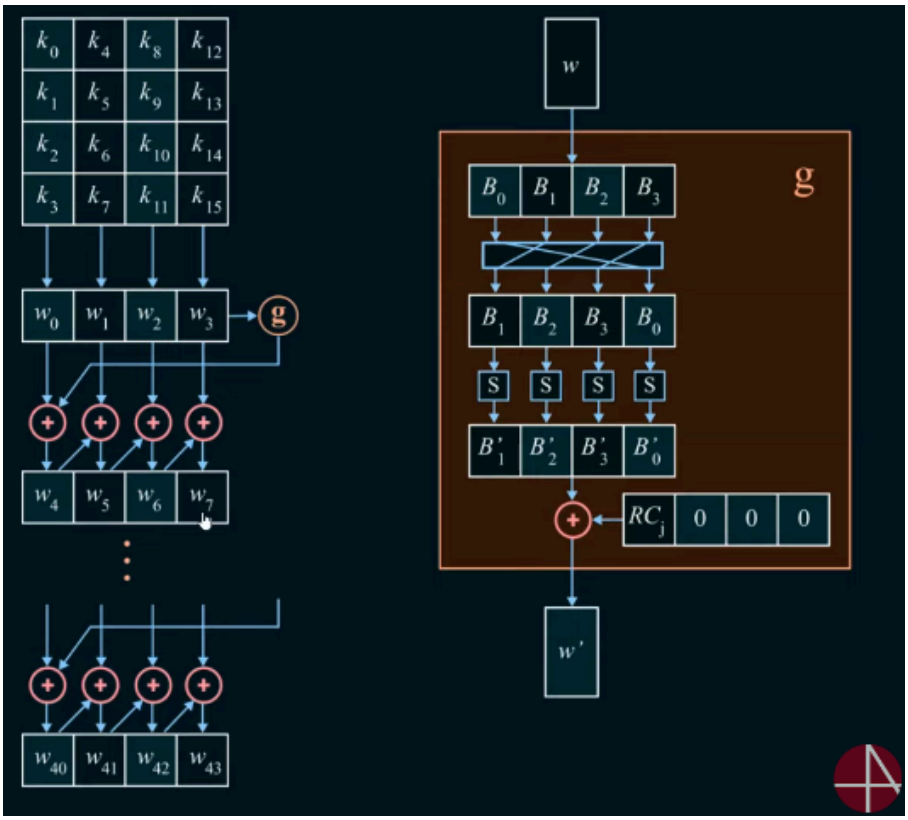


**Figure 3.1:** Key Expansion Logic

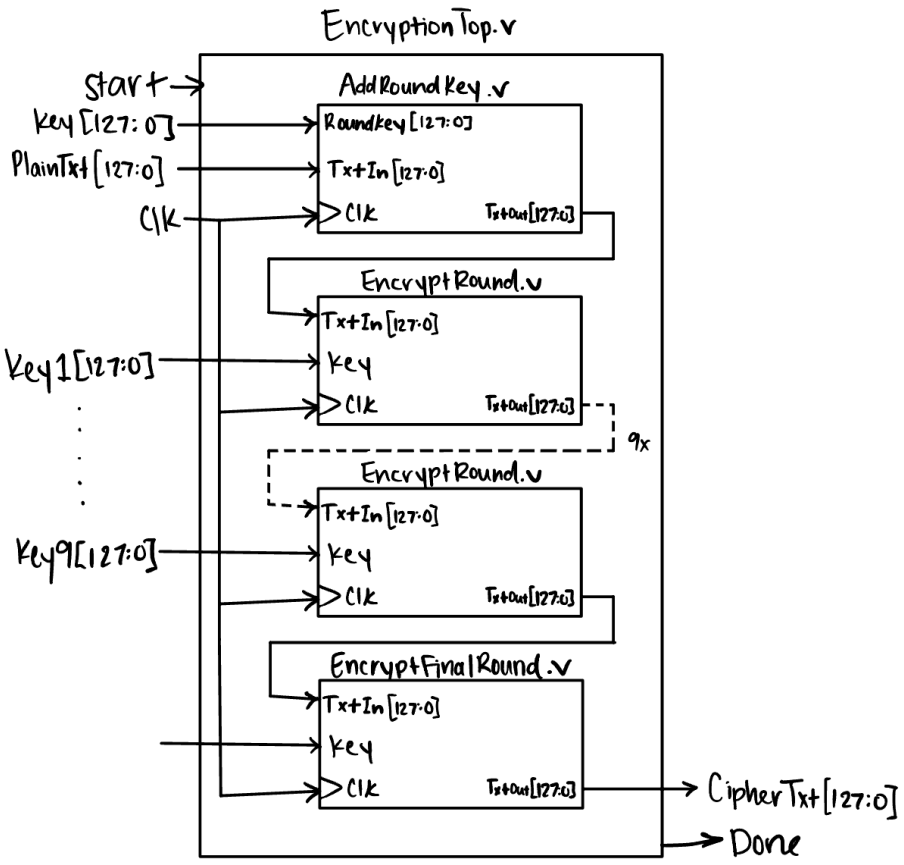


**Figure 3.2:** EncryptionTop.v Block Diagram

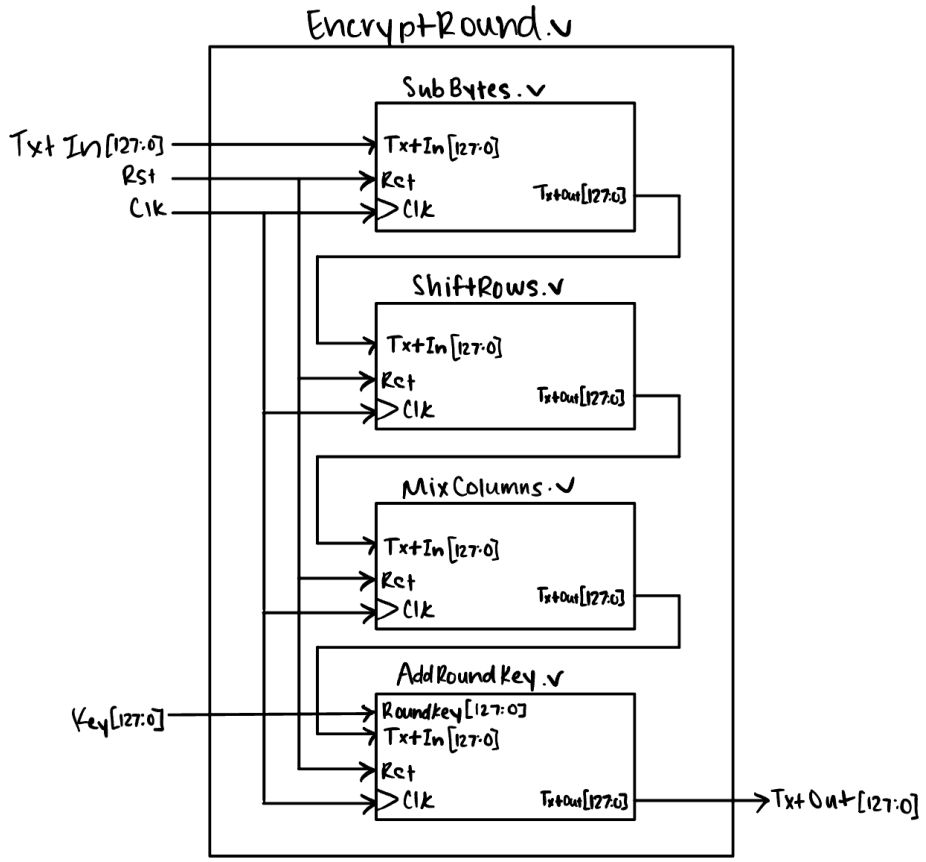


**Figure 3.3:** EncryptRound.v Block Diagram

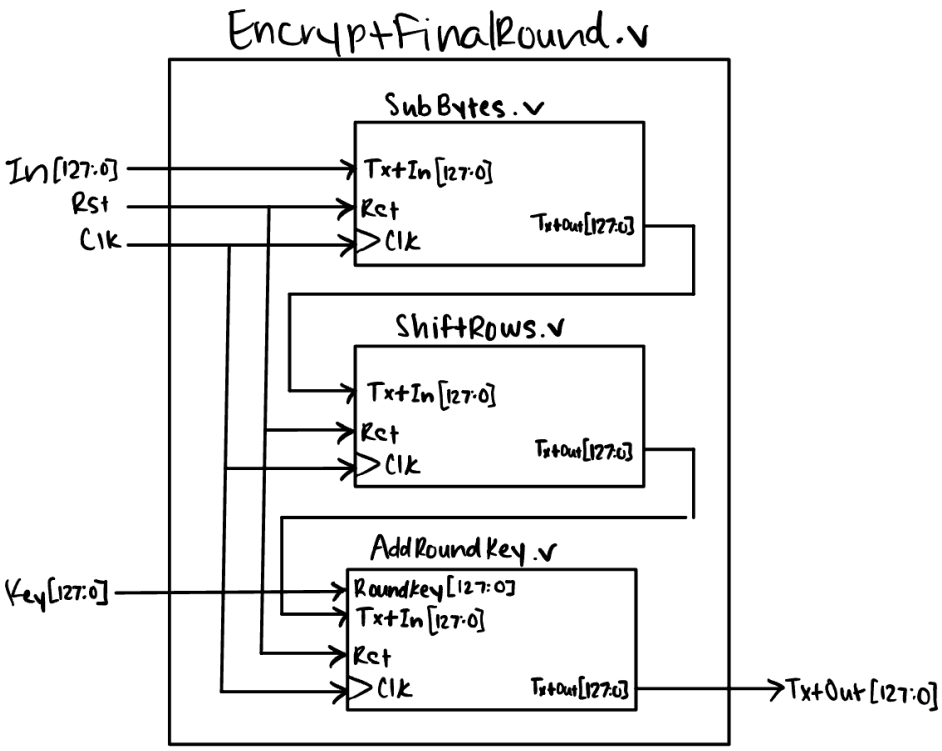


**Figure 3.4:** EncryptFinalRound.v Block Diagram

| | | Y | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | a | b | c | d | e | f |
| X | 0 | 63 | 7C | 77 | 7B | F2 | 6B | 6F | C5 | 30 | 01 | 67 | 2B | FE | D7 | AB | 76 |
| | 1 | CA | 82 | C9 | 7D | FA | 59 | 47 | F0 | AD | D4 | A2 | AF | 9C | A4 | 72 | C0 |
| | 2 | B7 | FD | 93 | 26 | 36 | 3F | F7 | CC | 34 | A5 | E5 | F1 | 71 | D8 | 31 | 15 |
| | 3 | 04 | C7 | 23 | C3 | 18 | 96 | 05 | 9A | 07 | 12 | 80 | E2 | EB | 27 | B2 | 75 |
| | 4 | 09 | 83 | 2C | 1A | 1B | 6E | 5A | A0 | 52 | 3B | D6 | B3 | 29 | E3 | 2F | 84 |
| | 5 | 53 | D1 | 00 | ED | 20 | FC | B1 | 5B | 6A | CB | BE | 39 | 4A | 4C | 58 | CF |
| | 6 | D0 | EF | AA | FB | 43 | 4D | 33 | 85 | 45 | F9 | 02 | 7F | 50 | 3C | 9F | A8 |
| | 7 | 51 | A3 | 40 | 8F | 92 | 9D | 38 | F5 | BC | B6 | DA | 21 | 10 | FF | F3 | D2 |
| | 8 | CD | 0C | 13 | EC | 5F | 97 | 44 | 17 | C4 | A7 | 7E | 3D | 64 | 5D | 19 | 73 |
| | 9 | 60 | 81 | 4F | DC | 22 | 2A | 90 | 88 | 46 | EE | B8 | 14 | DE | 5E | 0B | DB |
| | a | E0 | 32 | 3A | 0A | 49 | 06 | 24 | 5C | C2 | D3 | AC | 62 | 91 | 95 | E4 | 79 |
| | b | E7 | C8 | 37 | 6D | 8D | D5 | 4E | A9 | 6C | 56 | F4 | EA | 65 | 7A | AE | 08 |
| | c | BA | 78 | 25 | 2E | 1C | A6 | B4 | C6 | E8 | DD | 74 | 1F | 4B | BD | 8B | 8A |
| | d | 70 | 3E | B5 | 66 | 48 | 03 | F6 | 0E | 61 | 35 | 57 | B9 | 86 | C1 | 1D | 9E |
| | e | E1 | F8 | 98 | 11 | 69 | D9 | 8E | 94 | 9B | 1E | 87 | E9 | CE | 55 | 28 | DF |
| | f | 8C | A1 | 89 | 0D | BF | E6 | 42 | 68 | 41 | 99 | 2D | 0F | B0 | 54 | BB | 16 |

**Figure 3.5:** S-Box

| | | | |
|---|---|---|---|
| txt[127:120] | txt[95:88] | txt[63:56] | txt[31:24] |
| txt[119:112] | txt[87:80] | txt[55:48] | txt[25:16] |
| txt[111:104] | txt[79:72] | txt[47:40] | txt[15:8] |
| txt[103:96] | txt[71:64] | txt[39:32] | txt[7:0] |

**Table 3.6:** Matrix Organization

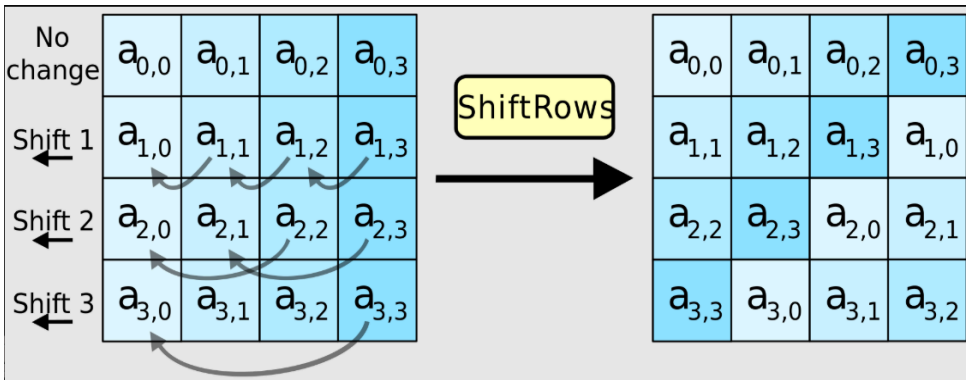


**Figure 3.7:** Shift Rows Logic

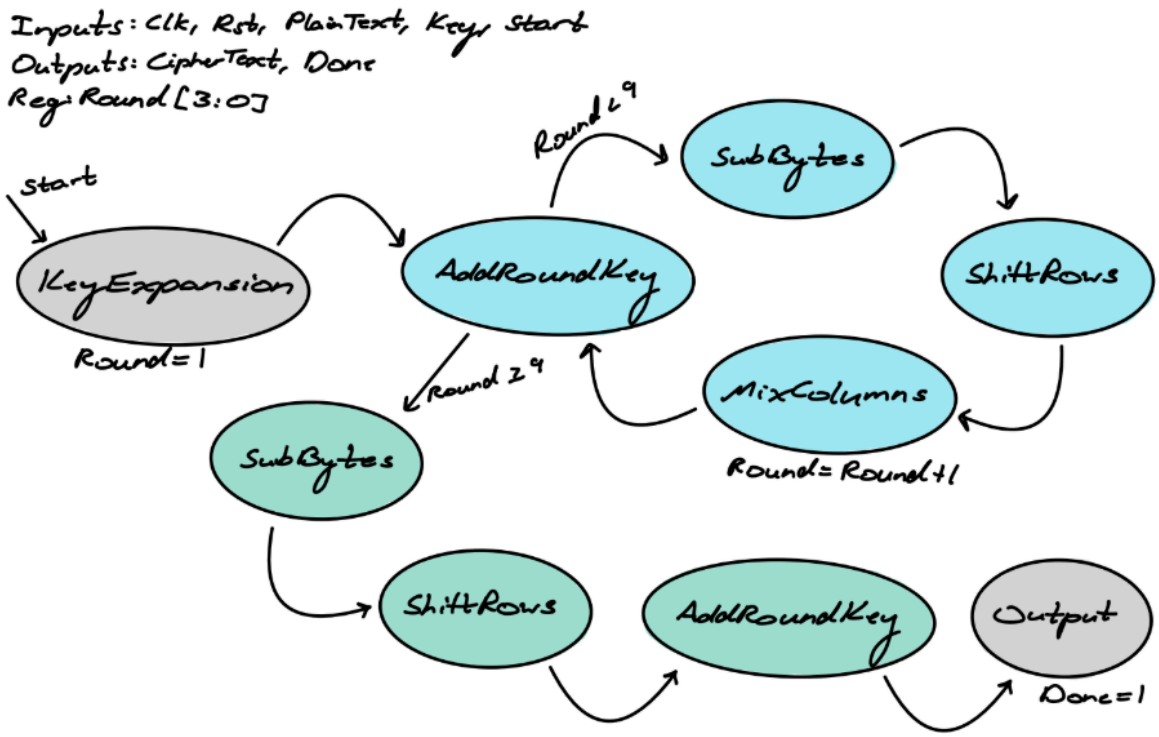


**Figure 3.8:** FSM Diagram for Encryption.v

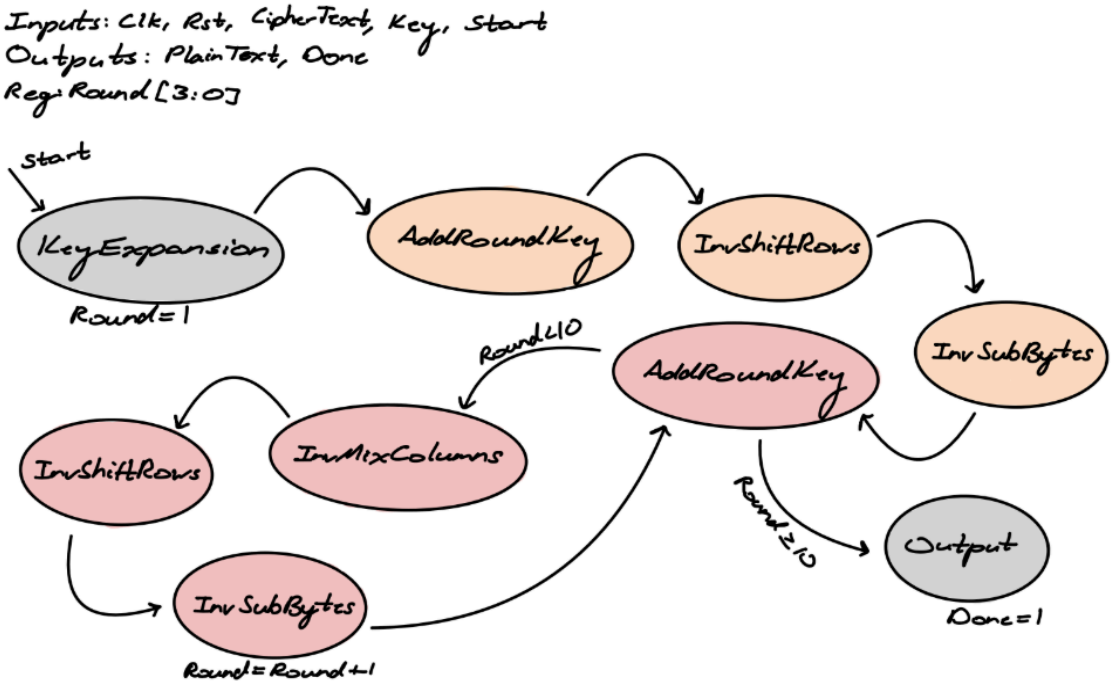


**Figure 3.9:** FSM Diagram for Decryption.v

$$\begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix}$$

**Figure 3.10:** Inverse Mix Columns Matrix

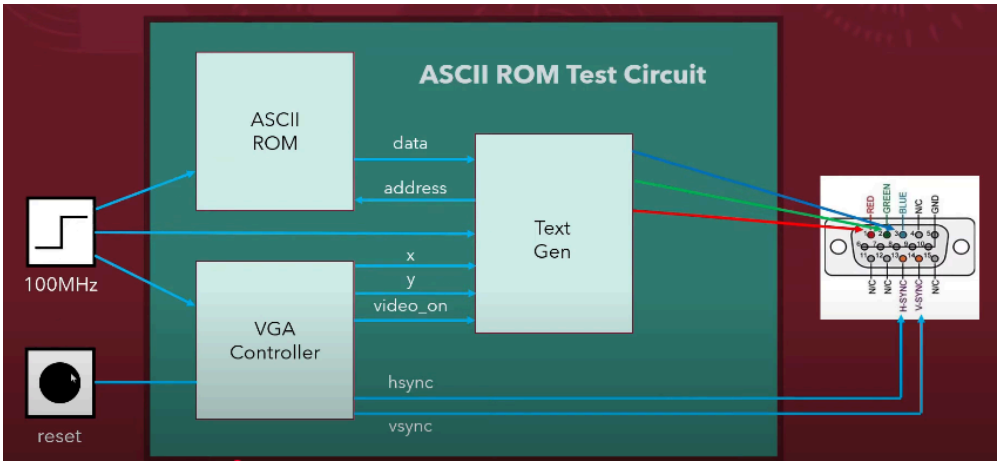


**Figure 4.1:** VGA Output Block Diagram

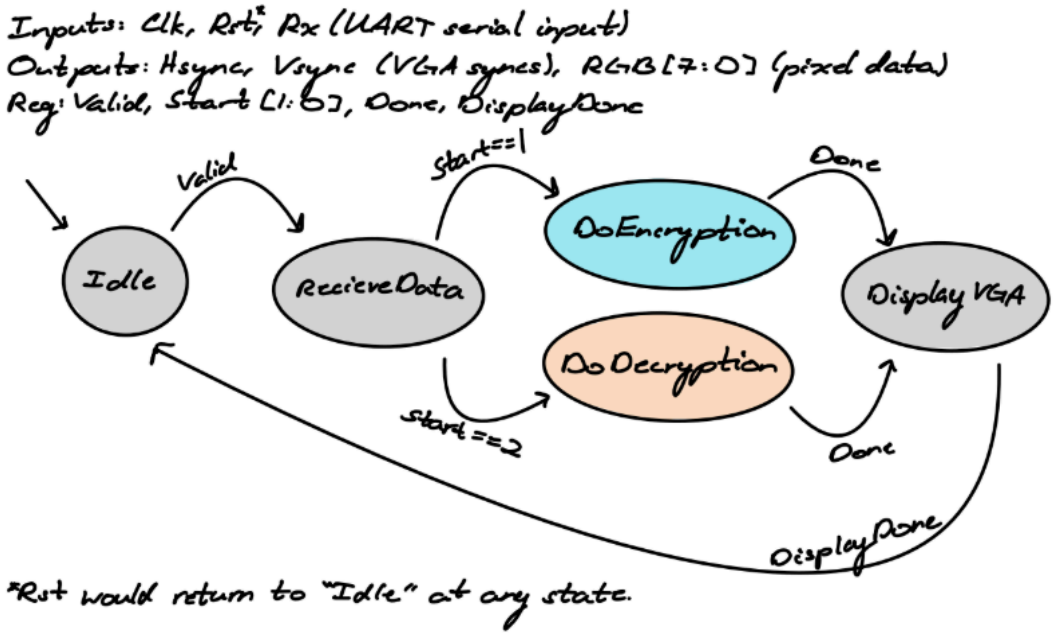


**Figure 5.1**: FSM diagram for Top.v

| Resource | Utilization | Available | Utilization % |
|---|---|---|---|
| LUT | 17919 | 63400 | 28.26 |
| LUTRAM | 6 | 19000 | 0.03 |
| FF | 660 | 126800 | 0.52 |
| BRAM | 0.50 | 135 | 0.37 |
| IO | 20 | 210 | 9.52 |
| BUFG | 3 | 32 | 9.38 |

**Figure 6.1**: Resource utilization

```
LXT2 info: dumpfile dump.lx2 opened for output.
Test done,           0 errors

InvShiftRows_tb.v:51: $finish called at 4000 (1ps)
```

**Figure 7.1:** Output Testbench from InvShiftRows_tb.v